



# Improve the Performance of Python with Profiling

Here are some key insights for Python users who want to speed up their code.

In most aspects of our life, speed is important—whether it is while driving or executing code. Since the dawn of computer science, we have argued about whether our code should use less memory or take less time to execute. Since the price of memory has been steadily decreasing, developers now focus on making their code run faster and spend precious hours trying to shave those few seconds off their execution time. But how does one go about trying to speed up working code? Well, you could just randomly tweak areas of code, but this method has proved to be messy and very inefficient. Usually, I prefer writing code in Python, but sadly when I get into an argument with someone about the performance of Python, the usual responses I get are “Python is too slow,” or “The global interpreter lock restricts Python’s code performance,” or something along those lines. So my goals for writing this article are as follows:

- Getting faster code using pure Python.
- To list some techniques and tools you can use without resorting to writing code in C or assembly.

Here is the list of libraries that the required tools depend on, so download and install them, if you haven’t already done so:

- CPython (The default interpreter)
- line\_profiler
- Runsnake

Let’s get started by looking at some code! This is the same code we will try to improve in subsequent portions of the article, and it’s the boring old matrix multiplication code (*matrix\_mult.py*):

```
import random
from time import *
import cProfile

class matrix(object):
    def __init__(self, m, n):
        # Create random matrix
        self.m=m
        self.n=n
        self.mat = [[random.random() for row in range(n)] for col
in range(m)]

    def zero(self):
        self.mat = [[0 for row in range(self.n)] for col in
range(self.m)]

    def mult(matrix1, matrix2):
        # Matrix multiplication
```